CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment – 5

Name: Prionti Das | **UID: 24BAI71005**

Branch: CSE(AIML) | **Section/Group: 24AIT_NTPP-3[B]**

Semester:5th | **Date of Performance: 13/08/2026**

Subject:Full Stack -II | **Subject Code: 24CSP-337**

EXPERIMENT – 5.1

1. **AIM:** To design and implement RESTful APIs using Spring Boot with proper validation, standardized responses, and scalable architecture.

2. **OBJECTIVE :**
   1. To understand the principles and architecture of RESTful APIs using Spring Boot.
   2. To implement CRUD operations using appropriate HTTP methods such as GET, POST, PUT, and DELETE.
   3. To apply input validation techniques to ensure correct and reliable API requests.
   4. To implement standardized API responses for consistent client-server communication.
   5. To implement exception handling for managing invalid requests and runtime errors.
   6. To understand and apply a layered architecture for developing scalable and maintainable applications.
   7. To test and verify RESTful APIs using tools such as Postman.

3. **SOFTWARE REQUIREMENTS:**
   1. Java JDK 17
   2. Spring Boot
   3. Visual Studio Code
   4. Postman

4. **THEORY :**

REST (Representational State Transfer) is an architectural style used to develop web services that allow communication between client and server over HTTP. RESTful APIs use standard HTTP methods such as **GET, POST, PUT, and DELETE** to perform different operations on resources.

Spring Boot is a Java-based framework that simplifies the development of web applications and RESTful APIs. It provides annotations such as `@RestController`, `@GetMapping`, `@PostMapping`, `@PutMapping`, and `@DeleteMapping` to create API endpoints easily.

API validation is used to ensure that the data received from the client is correct and complete. Spring Boot provides **Bean Validation** annotations such as `@NotBlank`, `@Email`, and `@Min` for validating request data.

Prionti_24BAI71005

A scalable REST application commonly follows a **layered architecture** consisting of the Controller, Service, Repository, and Model/DTO layers. Exception handling and standardized responses provide consistent error and success messages, making the application easier to maintain, test, and extend.

5. IMPLEMENTATION/PROCEDURE:

1. Develop a **Spring Boot CRUD application** for managing REST API posts.
2. Create a web interface named **REST API Post Manager** to allow users to create, update, view, and delete posts.
3. Design the **Create / Update Post** form with the following fields:
    - Title
    - Author
    - Scheduled Date & Time
    - Status
    - Content
4. Add a **Status** dropdown containing options such as `SCHEDULED` for selecting the current post status.
5. Implement **Save Post** functionality to submit the entered post details to the backend REST API.
6. Implement **Clear** functionality to reset the form fields and allow the user to enter new post details.
7. Implement backend **CRUD REST APIs** using Spring Boot to create, retrieve, update, and delete post records.
8. Add **input validation** to ensure that the required post information is entered correctly before saving the data.
9. Implement **standardized API responses** so that successful operations and errors are returned in a consistent format.
10. Display all saved posts in an **All Posts** section using a tabular format containing:
    - ID
    - Title
    - Author
    - Scheduled At
    - Status
    - Actions
11. Implement **Refresh** functionality to retrieve the latest post records from the backend and display them in the table.
12. Add **Edit** buttons to load an existing post's details into the Create / Update Post form for modification.
13. Add **Delete** buttons to remove the selected post from the database through the REST API.
14. Test the application by creating sample posts, updating existing posts, retrieving post records, and deleting posts.
15. Verify that the saved records are correctly displayed in the **All Posts** table with their corresponding ID, title, author, scheduled date and time, and status.

6. CODE :

Postservice.java :

```
package com.example.rest_api_experiment.service;
```

Prionti_24BAI71005

```
import java.util.List;

import org.springframework.stereotype.Service;

import com.example.rest_api_experiment.dto.PostRequest;
import com.example.rest_api_experiment.entity.Post;
import com.example.rest_api_experiment.exception.ResourceNotFoundException;
import com.example.rest_api_experiment.repository.PostRepository;

@Service
public class PostService {
  private final PostRepository postRepository;

   public PostService(PostRepository postRepository) {
      this.postRepository = postRepository;
   }

   public Post createPost(PostRequest request) {

      Post post = new Post();

      post.setTitle(request.getTitle());
      post.setContent(request.getContent());
      post.setAuthor(request.getAuthor());
      post.setScheduledAt(request.getScheduledAt());
      post.setStatus(request.getStatus());

      return postRepository.save(post);
   }

   public List<Post> getAllPosts() {

      return postRepository.findAll();
   }

   public Post getPostById(Long id) {

      return postRepository.findById(id)
            .orElseThrow(() ->
                  new ResourceNotFoundException(
```

Prionti_24BAI71005

```
                        "Post not found with id: " + id
                    )
                );
    }

    public Post updatePost(Long id, PostRequest request) {

        Post post = getPostById(id);

        post.setTitle(request.getTitle());
        post.setContent(request.getContent());
        post.setAuthor(request.getAuthor());
        post.setScheduledAt(request.getScheduledAt());
        post.setStatus(request.getStatus());

        return postRepository.save(post);
    }

    public void deletePost(Long id) {

        Post post = getPostById(id);

   postRepository.delete(post);
    }
}
```

ApiResponse.java:

```
package com.example.rest_api_experiment.response;

public class ApiResponse<T> {

    private boolean success;
    private String message;
    private T data;

    public ApiResponse() {
    }

    public ApiResponse(boolean success, String message, T data) {
        this.success = success;
```

Prionti_24BAI71005

```
        this.message = message;
        this.data = data;
    }

    public boolean isSuccess() {
        return success;
    }

    public void setSuccess(boolean success) {
        this.success = success;
    }

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }

    public T getData() {
        return data;
    }

    public void setData(T data) {
        this.data = data;
    }
}
```

Prionti_24BAI71005

7. OUTPUT:

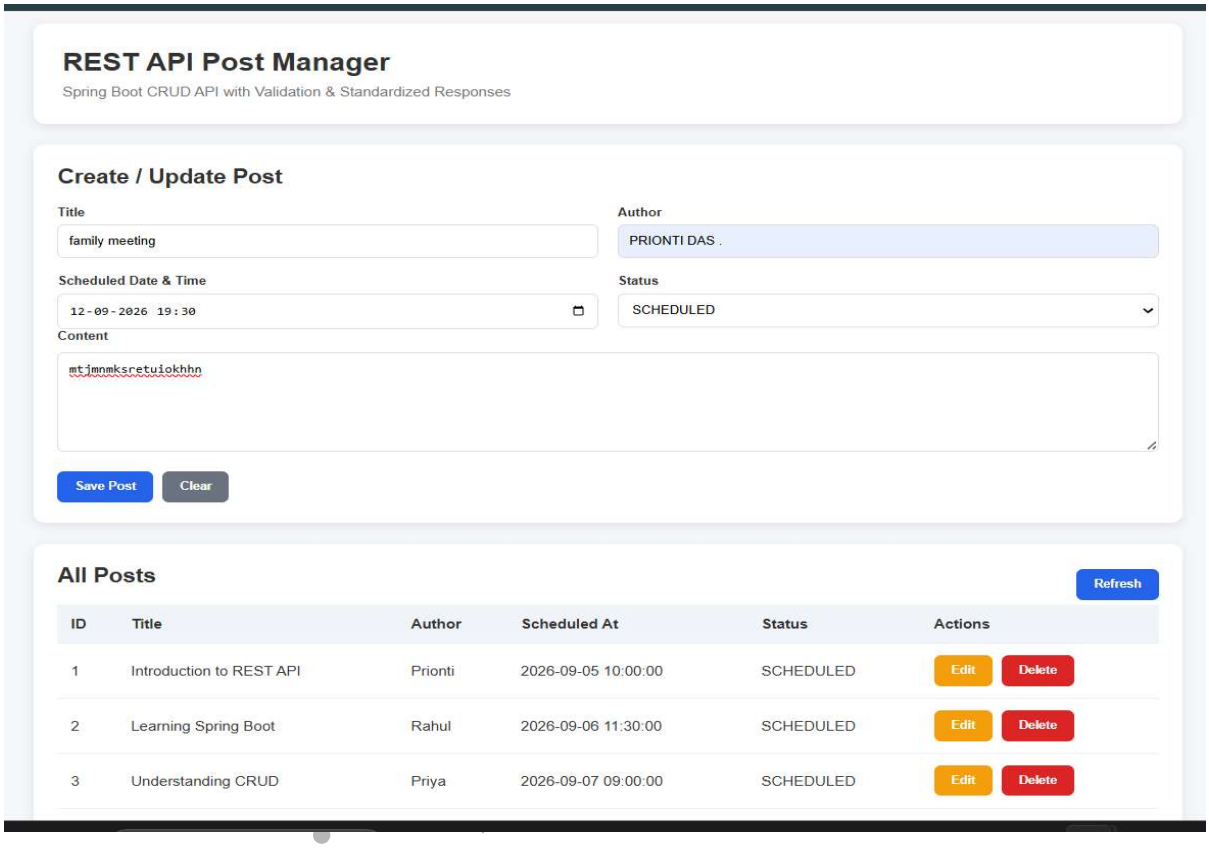


Fig 5.1: REST API Post Manager – Post Creation and Management Interface

The output shows the **REST API Post Manager** web application for managing posts. The form allows the user to enter the **title, author, scheduled date and time, status, and content** of a post. The **Save Post** button stores the post, while **Clear** resets the form. The **All Posts** table displays existing posts with their ID, title, author, scheduled time, and status. **Edit** and **Delete** buttons provide update and delete functionality, while **Refresh** retrieves the latest post records from the backend.

Prionti

8. LEARNING OUTCOME:

1. Design and develop RESTful APIs using Spring Boot and standard HTTP methods.
2. Implement CRUD operations for managing application data.
3. Apply input validation to ensure accurate and reliable API requests.
4. Implement standardized responses and centralized exception handling.
5. Understand layered architecture for building scalable and maintainable backend applications.
6. Test and verify REST APIs using Postman.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 5.2

1. **AIM:** To implement global exception handling and structured logging for building robust and observable backend systems.
2. **OBJECTIVE:**
    1. To understand the importance of global exception handling in Spring Boot applications.
    2. To implement centralized exception handling using `@ControllerAdvice` and `@ExceptionHandler`.
    3. To handle application and runtime exceptions in a consistent manner.
    4. To implement structured logging for monitoring and debugging backend activities.
    5. To use appropriate logging levels such as INFO, DEBUG, WARN, and ERROR.
    6. To generate standardized error responses for different types of exceptions.
    7. To improve system observability, debugging, and maintainability.
    8. To implement correlation IDs for tracing requests across the application.
3. **SOFTWARE REQUIREMENTS:**
    1. Java JDK 17
    2. Spring Boot
    3. Visual Studio Code
    4. Postman
4. **THEORY :**

Global exception handling is a mechanism used to manage exceptions from a centralized location in a Spring Boot application. It avoids writing separate exception-handling code in every controller and provides consistent error responses.

Spring Boot provides annotations such as `@ControllerAdvice` or `@RestControllerAdvice` and `@ExceptionHandler` for handling exceptions globally. Custom exceptions such as `ResourceNotFoundException` can be created for specific application errors.

Structured logging is used to record application events, requests, warnings, and errors in an organized format. Logging levels such as **INFO, DEBUG, WARN, and ERROR** help in monitoring and debugging the application.

A **correlation ID** is a unique identifier assigned to a request, which helps trace that request through different components of the backend system. Together, global exception handling and structured logging make the application more robust, observable, and easier to maintain.

5. **IMPLEMENTATION/PROCEDURE :**
    1. Create a Spring Boot application for managing student records and handling exceptions globally.
    2. Configure the required dependencies for building the REST API, database connectivity, and validation.
    3. Create a Student entity with the following fields:

Prionti_24BAI71005

- ID
- Name
- Email
- Course
- Age

4. Create a Student Repository to perform database operations such as saving, retrieving, and searching student records.
5. Create a Student Service to handle the application logic for adding and retrieving student details.
6. Create REST API endpoints through a Student Controller to perform student-related operations.
7. Design the Student API Dashboard web interface with an Add Student form containing Name, Email, Course, and Age fields.
8. Implement the Add Student button to send the entered student details to the backend API and store the record.
9. Create a Find Student section with an ID input field and provide Search and Show All functionalities.
10. Implement the Search functionality to retrieve a specific student record using the entered student ID.
11. Implement the Show All functionality to retrieve and display all available student records.
12. Display the retrieved student information in a Students table containing ID, Name, Email, Course, and Age.
13. Implement global exception handling using `@RestControllerAdvice` and `@ExceptionHandler` to handle errors such as a student not being found.
14. Configure appropriate logging in the application to record important events such as student creation, search requests, successful operations, and errors.
15. Run the Spring Boot application and access the Student API Dashboard through the web browser.
16. Add a sample student record and verify that it is successfully displayed in the Students table.
17. Enter a student ID in the Find Student section and use Search to verify retrieval of the required student record.
18. Use Show All to display all stored student records and verify that the API is working correctly.
19. Test invalid or non-existing student IDs to verify that global exception handling and error logging work correctly.
20. Capture the final dashboard and API output as the experiment result.

6. CODE :

StudentService.java:

```
package com.example.exception_logging_experiment.service;

import java.util.List;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

import com.example.exception_logging_experiment.dto.StudentRequest;
```

Prionti_24BAI71005

```
import com.example.exception_logging_experiment.entity.Student;
import com.example.exception_logging_experiment.exception.StudentNotFoundException;
import com.example.exception_logging_experiment.repository.StudentRepository;

@Service
public class StudentService {

    private static final Logger logger =
            LoggerFactory.getLogger(StudentService.class);

    private final StudentRepository studentRepository;

    public StudentService(StudentRepository studentRepository) {
        this.studentRepository = studentRepository;
    }

    public List<Student> getAllStudents() {

        logger.info("Fetching all students");

        return studentRepository.findAll();
    }

    public Student getStudentById(Long id) {

        logger.info("Fetching student with id={}", id);

        return studentRepository
                .findById(id)
                .orElseThrow(() ->
                        new StudentNotFoundException(
                                "Student not found with id: " + id
                        )
                );
    }

    public Student createStudent(StudentRequest request) {

        logger.info(
                "Creating student with email={}",
```

Prionti_24BAI71005

```
            request.getEmail()
        );

        Student student = new Student();

        student.setName(request.getName());
        student.setEmail(request.getEmail());
        student.setCourse(request.getCourse());
        student.setAge(request.getAge());

        return studentRepository.save(student);
    }

    public Student updateStudent(
            Long id,
            StudentRequest request) {

        logger.info("Updating student id={}", id);

        Student student = getStudentById(id);

        student.setName(request.getName());
        student.setEmail(request.getEmail());
        student.setCourse(request.getCourse());
        student.setAge(request.getAge());

        return studentRepository.save(student);
    }

    public void deleteStudent(Long id) {

        logger.info("Deleting student id={}", id);

        Student student = getStudentById(id);

        studentRepository.delete(student);

        logger.info(
            "Student deleted successfully id={}",
            id
```

Prionti_24BAI71005

```
        );
    }
}
```

7. OUTPUT:

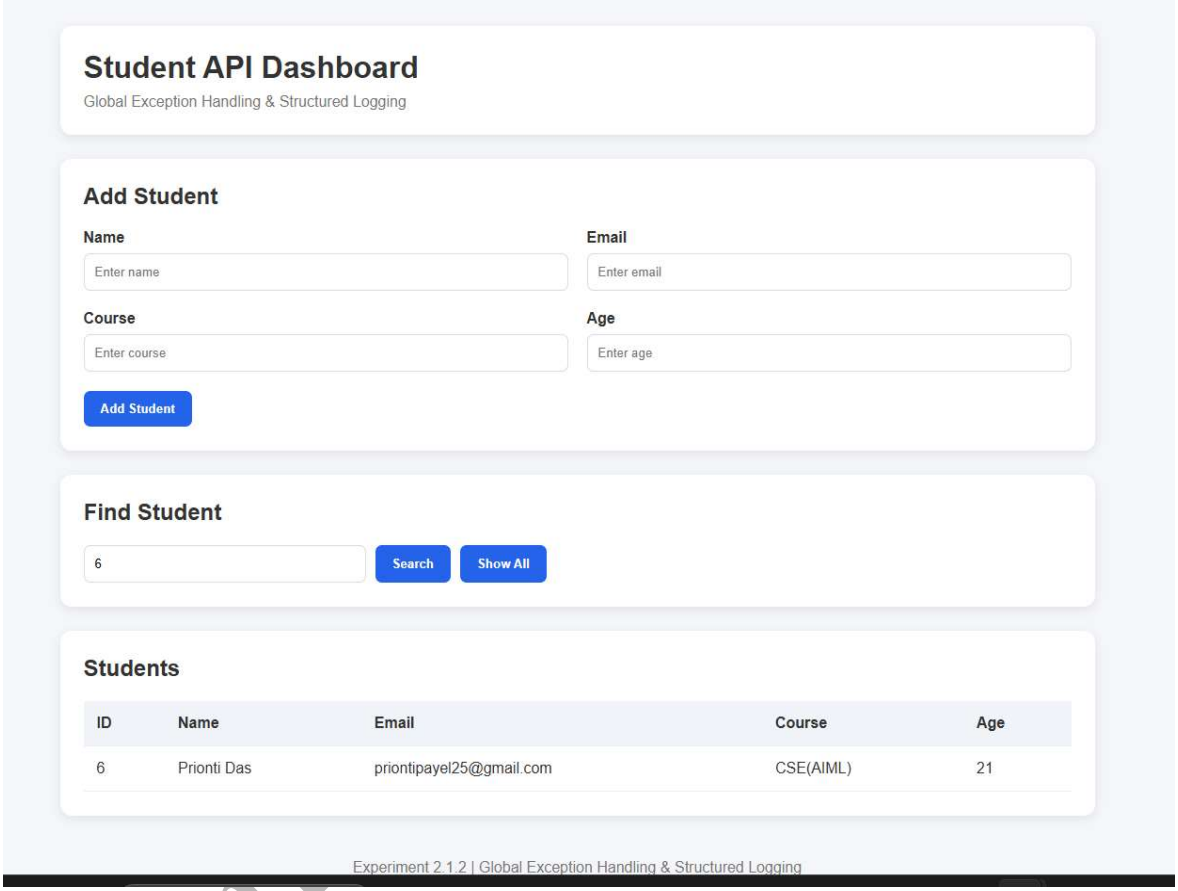


Fig 5.2: Student API Dashboard – Global Exception Handling and Structured Logging

The output shows a **Student API Dashboard** developed using Spring Boot. It allows users to add student details such as **Name, Email, Course, and Age**, search for a student using their ID, and display all student records in a table. The experiment also demonstrates **global exception handling** for invalid or unavailable student records and **structured logging** for tracking operations such as fetching, creating, updating, and deleting students.

8. LEARNING OUTCOME:

1. Implement global exception handling in a Spring Boot application.
2. Create custom exceptions and standardized error responses.
3. Implement structured logging using SLF4J and appropriate log levels.
4. Monitor and trace application requests and errors effectively.
5. Improve the robustness, observability, debugging, and maintainability of backend systems.